



‘Minería de datos y modelización predictiva’

Pablo Blas Hernández

1. Introducción

El objetivo principal de esta práctica es analizar la influencia de factores demográficos, socioeconómicos y territoriales en los resultados electorales de los municipios de España. El estudio se centra en el voto a partidos de derecha (PP y Ciudadanos) y se aborda desde una doble perspectiva analítica mediante dos variables objetivo:

- "Dcha_Pct" (variable continua): Porcentaje de votos a partidos de derecha (PP y Ciudadanos). Su naturaleza continua permite utilizar un modelo de regresión lineal para explorar cómo las variables explicativas influyen en este porcentaje.
- "Derecha" (variable dicotómica): Toma el valor 1 si la suma de los votos de derecha es superior a la de izquierda y otros y, 0, en otro caso. Al ser una variable binaria, se empleará un modelo de regresión logística para identificar qué variables predictoras aumentan o disminuyen la probabilidad de que un municipio tenga una mayoría de votos de derecha.

El conjunto de variables explicativas incluye indicadores demográficos como población, densidad y crecimiento poblacional; variables económicas relacionadas con la actividad predominante, los servicios y el comercio; variables habitacionales asociadas a las condiciones de vida; y variables territoriales vinculadas a la estructura productiva y al tamaño del municipio. Estas variables permiten capturar distintas dimensiones estructurales que pueden estar relacionadas con las preferencias políticas locales. Asimismo, siguiendo las especificaciones de la práctica, se excluirán del conjunto de variables explicativas el resto de las variables objetivo no seleccionadas (la abstención o el voto a otros bloques) para evitar sesgos y garantizar la independencia del análisis.

La metodología del estudio comprende un análisis exploratorio de datos (EDA) para detectar patrones, valores atípicos y problemas de calidad de los datos, seguido de un proceso de limpieza, tratamiento de valores atípicos e imputación de datos faltantes. Posteriormente, se construyen los modelos de regresión lineal y logística aplicando técnicas de selección de variables para identificar los predictores más relevantes. Finalmente, se evalúa el desempeño de los modelos y se interpretan los resultados para extraer conclusiones sustantivas.

En conjunto, este trabajo no solo busca predecir los resultados electorales municipales, sino también aportar una interpretación estadística y empírica de los factores socioeconómicos que subyacen al comportamiento electoral en España.

2. Importación y Análisis inicial

El primer paso en cualquier proyecto de minería de datos es importar las librerías que va a requerir nuestro proyecto. En este caso, importamos todo el contenido del archivo `FuncionesMineria.py`, de la siguiente manera: `from FuncionesMineria import *`. El conjunto de datos que utilizaremos es "DatosEleccionesEspaña.xlsx", que se importó utilizando la función `pd.read_excel()` de `pandas`. A continuación, eliminamos las variables objetivo que no forman parte del estudio y, además, se elimina la variable

‘CodigoProvincia’ por no aportar información relevante, y se establece la variable ‘Name’ como índice. El siguiente paso es verificar los tipos de datos asignados mediante `df.dtypes`, identificando la siguiente inconsistencia: ‘Derecha’ (variable binaria) se importa como numérica (`int64`), pero debe tratarse como categoría nominal, por lo que se corrige convirtiéndola a tipo `string`.

CCAA	object
Population	int64
TotalCensus	int64
Dcha_Pct	float64
Derecha	int64
Age_0-4_Ptge	float64
Age_under19_Ptge	float64
Age_19_65_pct	float64
Age_over65_pct	float64
WomanPopulationPtge	float64
ForeignersPtge	float64
SameComAutonPtge	float64
SameComAutonDiffProvPtge	float64
DifComAutonPtge	float64
UnemployLess25_Ptge	float64
Unemploy25_40_Ptge	float64
UnemployMore40_Ptge	float64
AgricultureUnemploymentPtge	float64
IndustryUnemploymentPtge	float64
ConstructionUnemploymentPtge	float64
ServicesUnemploymentPtge	float64
totalEmpresas	float64
Industria	float64
Construccion	float64
ComercTTEHosteleria	float64
Servicios	float64
ActividadPpal	object
inmuebles	float64
Pob2010	float64
SUPERFICIE	float64
Densidad	object
PobChange_pct	float64
PersonasInmueble	float64
Explotaciones	int64
dtype:	object

```
# Convertimos la variable 'Derecha' a categórica
numerica_a_categoria = ['Derecha']

for var in numerica_a_categoria :
    df[var] = df[var].astype(str)
```

3. Análisis descriptivo

El análisis descriptivo se dividirá en variables numéricas y categóricas.

3.1. Numéricas

```
numericas = df.select_dtypes(include=['number']).columns

estadisticos_var_num = df.describe().T

# Calculamos medidas extra
for num in numericas:
    estadisticos_var_num.loc[num, "Asimetria"] = df[num].skew()
    estadisticos_var_num.loc[num, "Kurtosis"] = df[num].kurtosis()
    estadisticos_var_num.loc[num, "Rango"] = np.ptp(df[num].dropna().values)

estadisticos_var_num
```

	count	mean	std	min	25%	50%	75%	max	Asimetria	Kurtosis	Rango
Population	8117.0	5722.344709	46204.175926	5.0000	166.0000	548.0000	2427.0000	3141991.000	46.040721	2820.326827	3.141986e+06
TotalCensus	8117.0	4247.863989	34423.439346	5.0000	140.0000	447.0000	1843.0000	2363829.000	46.544648	2893.452945	2.363824e+06
Dcha_Pct	8117.0	48.912166	19.946471	0.0000	38.6900	51.579	62.1870	100.000	-0.467616	-0.175615	1.000000e+02
Age_0_4_Ptge	8117.0	3.018274	2.052625	0.0000	1.3890	2.975	4.5330	13.245	0.343639	-0.206688	1.324500e+01
Age_under19_Ptge	8117.0	13.564087	6.777445	0.0000	8.3340	13.881	19.0550	33.696	-0.104763	-0.792250	3.369600e+01
Age_19_65_pct	8117.0	57.370592	6.818637	23.4590	53.8450	58.655	61.8180	100.002	-0.814264	2.155839	7.654300e+01
Age_over65_pct	8117.0	29.065321	11.767040	-18.0520	19.8270	27.559	36.9110	76.472	0.584708	0.102323	9.452400e+01
WomanPopulationPtge	8117.0	47.302297	4.362347	11.7650	45.7250	48.485	50.0000	72.683	-1.671102	5.800629	6.091800e+01
ForeignersPtge	8117.0	5.618322	7.348700	-8.9600	1.0600	3.590	8.1800	71.470	2.498262	11.356789	8.043000e+01
SameComAutonPtge	8117.0	81.633460	12.287323	0.0000	75.8060	84.493	90.4620	127.156	-1.522758	3.479538	1.271560e+02
SameComAutonDiffProvPtge	8117.0	4.337637	6.394942	0.0000	0.6760	2.190	5.2770	67.308	3.286828	14.560136	6.730800e+01
DiffComAutonPtge	8117.0	10.727261	8.847631	0.0000	4.9330	8.269	13.8910	100.000	2.425991	9.663965	1.000000e+02
UnemployLess25_Ptge	8117.0	7.320241	9.408801	0.0000	0.0000	5.882	10.4670	100.000	4.150962	31.664819	1.000000e+02
Unemploy25_40_Ptge	8117.0	37.001267	20.319055	0.0000	28.5710	39.927	46.6670	100.000	0.213481	1.412091	1.000000e+02
UnemployMore40_Ptge	8117.0	55.678492	22.087672	0.0000	44.1710	52.000	64.5830	100.000	0.259701	0.705886	1.000000e+02
AgricultureUnemploymentPtge	8117.0	8.402873	12.959440	0.0000	0.0000	3.497	11.7410	100.000	3.228916	15.572833	1.000000e+02
IndustryUnemploymentPtge	8117.0	10.009590	12.529462	0.0000	0.0000	7.143	14.2860	100.000	3.089440	16.047235	1.000000e+02
ConstructionUnemploymentPtge	8117.0	10.838410	13.282677	0.0000	0.0000	8.333	14.2860	100.000	3.093595	14.620223	1.000000e+02
ServicesUnemploymentPtge	8117.0	58.646833	24.261857	0.0000	50.0000	62.000	72.1310	100.000	-0.805605	0.800001	1.000000e+02
totalEmpresas	8112.0	397.701430	4219.492916	0.0000	7.0000	30.000	147.0000	299397.000	53.713554	3475.479731	2.993970e+05
Industria	7929.0	23.405347	158.628346	0.0000	0.0000	0.000	14.0000	10521.000	44.270924	2643.854715	1.052100e+04
Construccion	7978.0	48.811482	421.895011	0.0000	0.0000	0.000	25.0000	30343.000	52.577422	3506.443836	3.034300e+04
ComercioTTEHosteleria	8108.0	146.208806	1232.713016	0.0000	0.0000	0.000	65.0000	80856.000	45.459601	2652.634653	8.085600e+04
Servicios	8055.0	171.849783	2447.041982	0.0000	0.0000	0.000	40.0000	177677.000	57.502990	3833.616264	1.776770e+05
inmuebles	7979.0	3240.038225	24314.681445	6.0000	180.0000	485.000	1586.5000	1615548.000	44.561466	2646.689444	1.615542e+06
Pob2010	8110.0	5777.929840	47527.883258	5.0000	177.2500	581.500	2482.7500	3273049.000	47.200624	2944.830055	3.273044e+06
SUPERFICIE	8109.0	6215.296238	9218.604158	2.5784	1839.2392	3488.552	6894.5272	175022.910	6.073326	62.335015	1.750203e+05
PobChange_pct	8110.0	-4.900726	10.382355	-52.2700	-10.4000	-4.965	0.0900	138.460	1.506444	15.112052	1.907300e+02
PersonasInmueble	7979.0	1.295565	0.565993	0.1100	0.8500	1.250	1.7300	3.330	0.259748	-0.645897	3.220000e+00
Explotaciones	8117.0	2447.806456	15064.545054	1.0000	22.0000	52.000	137.0000	99999.000	6.321265	37.977123	9.999800e+04

Variables demográficas:

- La población media por municipio es de 5722 habitantes, pero con una gran dispersión (DE ± 46204). Esto refleja la predominancia de municipios pequeños (mediana = 548 hab.) frente a grandes ciudades como Madrid (3.14M hab.), lo que explica la asimetría extrema (46.04).
- El 29.06% de la población media es mayor de 65 años ('Age_over65_pct'), pero se detectaron valores imposibles (-18.05%), lo que sugiere errores en los datos.

Variables políticas:

- 'Dcha_Pct' muestra una media del 48.91% (DE ± 19.95), con distribución casi simétrica (asimetría -0.47). Sin embargo, el rango completo (0-100) indica municipios con dominancia absoluta de un bloque político.

Variables económicas:

- Servicios domina el empleo (58.65% del desempleo registrado), pero con alta variabilidad (DE ± 24.26).
- El número medio de empresas por municipio es 398, pero el 75% tiene menos de 147, frente a valores extremos como 299,397 en grandes ciudades (asimetría 53.71).

Variables habitacionales:

- 'ForeignersPtge' al igual que cómo hemos mencionado previamente con 'Age_over65_pct' incluye valores negativos (-8.96%), claramente erróneos.
- Explotaciones tiene un valor máximo de 99.999, que codifica datos faltantes.
- Age_19_65_pct registra un valor máximo ligeramente superior al 100%, un valor erróneo, pero que puede haber sido fruto de redondeo decimal, ya que es posible

que un municipio, como un pueblo de zona rural con pocos habitantes, cuente solo con habitantes adultos de entre 19 y 65 años.

3.2. Categóricas

```

categoricas = df.select_dtypes(include='object').columns
estadisticos_var_cat = {}
for categoria in categoricas:
    if categoria in df.columns:
        counts = df[categoria].value_counts()
        resultado = pd.DataFrame({
            'n': counts,
            '%': (counts / counts.sum() * 100).round(2).astype(str) + '%'
        })
        estadisticos_var_cat[categoria] = resultado

tabla_final = pd.concat(estadisticos_var_cat, axis=0)
tabla_final.index.names = ['Variable', 'Categoría']
tabla_final

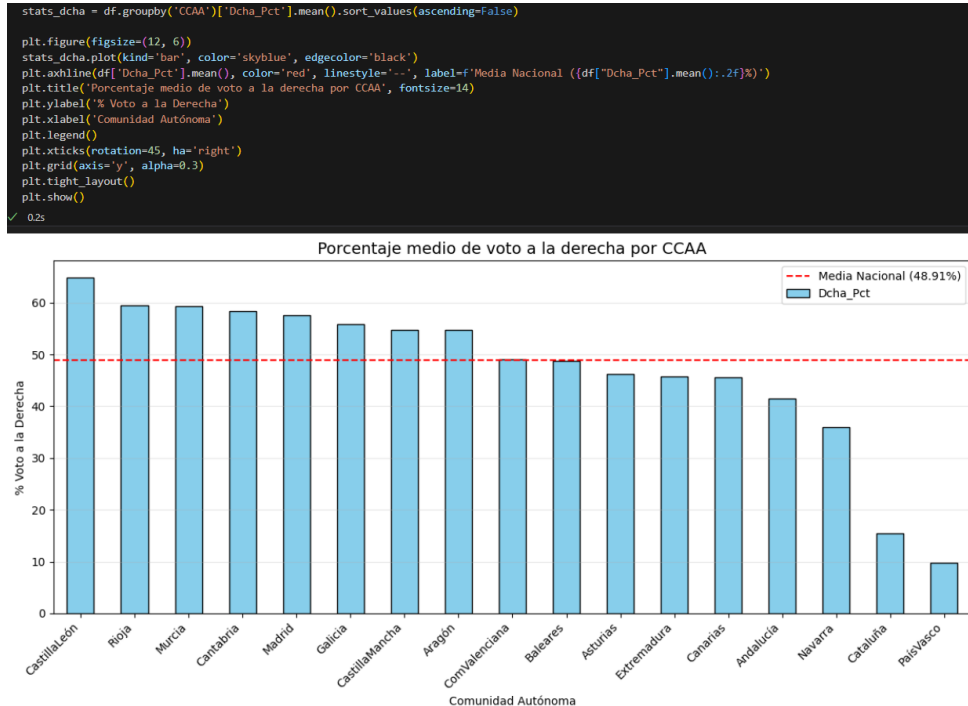
```

Variable	Categoría	n	%
CCAA	CastillaLeón	2248	27.69%
	Cataluña	947	11.67%
	CastillaMancha	919	11.32%
	Andalucía	773	9.52%
	Aragón	731	9.01%
	ComValenciana	542	6.68%
	Extremadura	387	4.77%
	Galicia	314	3.87%
	Navarra	272	3.35%
	PaísVasco	251	3.09%
	Madrid	179	2.21%
	Rioja	174	2.14%
	Cantabria	102	1.26%
	Canarias	88	1.08%
Derecha	Asturias	78	0.96%
	Baleares	67	0.83%
	Murcia	45	0.55%
	1	5041	62.1%
	0	3076	37.9%
ActividadPpal	Otro	4932	60.76%
	ComercTTEHosteleria	2538	31.27%
	Servicios	620	7.64%
	Construccion	14	0.17%
	Industria	13	0.16%
Densidad	MuyBaja	6416	79.04%
	Baja	1053	12.97%
	Alta	556	6.85%
	?	92	1.13%

En la variable ‘ActividadPpal’ hay categorías con poca representación como ‘Construcción’ e ‘Industria’ (<0.2% cada una). Estas deberían ser agrupadas para que tengan mayor representación.

Existen valores faltantes en la variable ‘Densidad’, que están representados como “?”. Estos valores deberán ser tratados posteriormente.

Castilla y León (27.69%), Cataluña (11.67%) y Castilla La Mancha (11.32%) son las comunidades autónomas con más representación en el dataset. Asturias, Baleares y Murcia son las CCAA que tienen menos representación (<1% cada una). Estos resultados son esperados debido a la superficie de cada CCAA y su número de municipios, ya que las siguientes con más representación son Andalucía y Aragón, que también abarcan bastante superficie en España. Hay muchas CCAA que no cuentan con una representación adecuada, por lo que será conveniente agruparlas. Para agruparlas, como nuestro objetivo de estudio es el voto a la derecha, vamos a tener en cuenta la predisposición de cada CCAA con el voto a la derecha:



4. Corrección de los errores detectados

Los principales problemas identificados en el análisis exploratorio se han corregido mediante las siguientes acciones:

Agrupaciones de las categorías ‘Construcción’ e ‘Industria’ de la variable ‘ActividadPpal’ a ‘Otro’, y las categorías de la variable ‘CCAA’ en base a su predisposición de voto a la derecha, como hemos comentado en el apartado anterior:

```
df['ActividadPpal'] = df['ActividadPpal'].replace({'Construccion': 'Otro', 'Industria': 'Otro'})
```

```
df['CCAA'] = df['CCAA'].replace({
    # Derecha muy alta (57-59%)
    'Rioja': 'Rio-Mur-Can-Mad',
    'Murcia': 'Rio-Mur-Can-Mad',
    'Cantabria': 'Rio-Mur-Can-Mad',
    'Madrid': 'Rio-Mur-Can-Mad',

    # Derecha consolidada (54-56%)
    'Galicia': 'Gal-CM-Ara',
    'Castilla-La Mancha': 'Gal-CM-Ara',
    'Aragón': 'Gal-CM-Ara',

    # Bloque intermedio (45-49%)
    'Comunidad Valenciana': 'CV-Bal-Ast-Ext-IC',
    'Baleares': 'CV-Bal-Ast-Ext-IC',
    'Asturias': 'CV-Bal-Ast-Ext-IC',
    'Extremadura': 'CV-Bal-Ast-Ext-IC',
    'Canarias': 'CV-Bal-Ast-Ext-IC',

    # Mínimos nacionales (9-15%)
    'Cataluña': 'Cat-PV',
    'País Vasco': 'Cat-PV'
})
```

Tratamiento de los valores “?” en la variable ‘Densidad’, que son reemplazados por NaN. El mismo proceso se repite en la variable ‘Explotaciones’ con los valores “99999”.

```
df['Densidad'] = df['Densidad'].replace('?', np.nan)
df['Explotaciones'] = df['Explotaciones'].replace(99999, np.nan)
```

Se eliminan los porcentajes negativos de las variables 'Age_over65_pct', 'ForeignersPtge', y los que superan el 100% de 'SameComAutonPtge', ya que son erróneos. No puede haber un porcentaje negativo de la población mayor a 65 años ni de la población extranjera, en todo caso su mínimo será 0%, pero no negativo. No puede haber un porcentaje superior a 100% de la población que reside en la misma CCAA de la que nacieron, en todo caso, su máximo será 100%, pero no más.

```
df['Age_over65_pct'] = [x if 0 <= x <= 100 else np.nan for x in df['Age_over65_pct']]
df['ForeignersPtge'] = [x if 0 <= x <= 100 else np.nan for x in df['ForeignersPtge']]
df['SameComAutonPtge'] = [x if 0 <= x <= 100 else np.nan for x in df['SameComAutonPtge']]
```

Se detectó una redundancia estructural en las variables de edad: la variable 'Age_under19_Ptge' incluía el subconjunto 'Age_0-4_Ptge', lo que generaba multicolinealidad perfecta y hacía que la suma total de porcentajes superase el 100%. En lugar de eliminar información, se opta por crear una nueva variable 'Age_5_18_Ptge' obtenida mediante la resta del porcentaje de 0-4 años ('Age_0-4_Ptge') al del grupo de menores de 19 ('Age_under19_Ptge'). De esta forma, descomponemos la población joven en dos segmentos disjuntos, y eliminamos la variable original 'Age_under19_Ptge'. Esto mejora la interpretabilidad del modelo y corrige la consistencia matemática de los datos. Además, se elimina la variable 'Age_19_65_pct' ya que podemos obtener sus datos a partir de las otras variables de edad, y la tomaremos como grupo referencia ('Age_19_65_pct' = 100 - 'Age_0-4_Ptge' - 'Age_5_18_Ptge' - 'Age_over65_pct'):

```
# (Under19 ya incluye a 0-4, así que no lo sumamos doble)
df['check_sum'] = df['Age_under19_Ptge'] + df['Age_19_65_pct'] + df['Age_over65_pct']

# Opcional: Filtramos solo errores reales muy graves (ej: suma > 101% por error de redondeo).
df = df[(df['check_sum'] >= 99.0) & (df['check_sum'] <= 101.0)]

# Creamos el tramo de "Niños en edad escolar" (5 a 18 años) restando los bebés al grupo de menores de 19.
df['Age_5_18_Ptge'] = df['Age_under19_Ptge'] - df['Age_0-4_Ptge']

# Eliminamos 'Age_under19_Ptge' porque ya tenemos sus dos componentes desglosados (0-4 y 5-18)
# Esto evita la multicolinealidad perfecta.
df.drop(columns=['Age_under19_Ptge', 'check_sum'], inplace=True)

# Eliminamos 'Age_19_65_pct' ya que podemos obtener su valor a partir de los demás grupos de edad
df.drop(columns=['Age_19_65_pct'], inplace=True)
```

5. Análisis de valores atípicos

Para detectar los valores atípicos de las variables se aplicó la siguiente función basada en el IQR a cada una de ellas:

```
def analizar_atipicos_con_nombres(df, col, umbral=1.5, n_ejemplos=3):
    """
    Calcula outliers por IQR y muestra los nombres de los municipios (usando el índice).
    """
    serie = df[col].dropna()
    if serie.empty: return

    # Cálculo de límites (IQR)
    Q1 = serie.quantile(0.25)
    Q3 = serie.quantile(0.75)
    IQR = Q3 - Q1
    lim_inf = Q1 - umbral * IQR
    lim_sup = Q3 + umbral * IQR

    outliers = df[(df[col] < lim_inf) | (df[col] > lim_sup)]

    if not outliers.empty:
        print(f"\n('='*50)=")
        print(f" VARIABLE: {col}")
        print(f"{'='*50}")
        print(f"Total outliers: {len(outliers)} ({len(outliers)/len(df)*100:.2f}%)")
        print(f"Límites IQR ({umbral}x): [{lim_inf:.2f}, {lim_sup:.2f}]")

        # Ejemplos de valores ALTOS (Por encima del límite superior)
        altos = outliers[outliers[col] > lim_sup].sort_values(by=col, ascending=False).head(n_ejemplos)
        if not altos.empty:
            print(f"Ejemplos MÁS ALTOS:")
            for idx, row in altos.iterrows():
                print(f"    - {idx}: {row[col]:.2f}")

        # Ejemplos de valores BAJOS (Por debajo del límite inferior)
        bajos = outliers[outliers[col] < lim_inf].sort_values(by=col, ascending=True).head(n_ejemplos)
        if not bajos.empty:
            print(f"Ejemplos MÁS BAJOS:")
            for idx, row in bajos.iterrows():
                print(f"    - {idx}: {row[col]:.2f}")

    cols_numericas = df.select_dtypes(include=[np.number]).columns

    # Excluimos columnas target
    cols_excluir = ['Dcha_Pct', 'Derecha']
    cols_analisis = [c for c in cols_numericas if c not in cols_excluir]

    print("INFORME DE VALORES ATÍPICOS (Con Nombres del Índice)")

    for col in cols_analisis:
        analizar_atipicos_con_nombres(df, col)
```

VARIABLE: Population Total outliers: 1154 (14.22%) Límites IQR (1.5x): [-3225.50, 5818.50] Ejemplos MÁS ALTOS: - Madrid: 3141991.00 - Barcelona: 1684555.00 - Valencia: 786189.00	VARIABLE: SameComAutonDiffProvPtge Total outliers: 675 (8.32%) Límites IQR (1.5x): [-6.23, 12.18] Ejemplos MÁS ALTOS: - Riu de Cerdanya: 67.31 - Riells i Viabrea: 57.92 - Querol: 54.53	VARIABLE: IndustryUnemploymentPtge Total outliers: 247 (3.04%) Límites IQR (1.5x): [-21.43, 35.71] Ejemplos MÁS ALTOS: - Sacañet: 100.00 - Alcalá de la Vega: 100.00 - Garaballa: 100.00	VARIABLE: Servicios Total outliers: 1274 (15.70%) Límites IQR (1.5x): [-60.00, 100.00] Ejemplos MÁS ALTOS: - Madrid: 177677.00 - Barcelona: 103010.00 - Valencia: 34526.00
VARIABLE: TotalCensus Total outliers: 1149 (14.16%) Límites IQR (1.5x): [-2414.50, 4397.50] Ejemplos MÁS ALTOS: - Madrid: 2363829.00 - Barcelona: 1136354.00 - Valencia: 578722.00	VARIABLE: DifComAutonPtge Total outliers: 388 (4.78%) Límites IQR (1.5x): [-8.50, 27.33] Ejemplos MÁS ALTOS: - Illán de Vacas: 100.00 - Ugena: 75.26 - El Viso de San Juan: 73.07	VARIABLE: ConstructionUnemploymentPtge Total outliers: 343 (4.23%) Límites IQR (1.5x): [-21.43, 35.71] Ejemplos MÁS ALTOS: - Villamiel de la Sierra: 100.00 - Bascuñana de San Pedro: 100.00 - Arrancacepas: 100.00	VARIABLE: Inmuebles Total outliers: 1050 (12.94%) Límites IQR (1.5x): [-1929.75, 3696.25] Ejemplos MÁS ALTOS: - Madrid: 161548.00 - Barcelona: 857862.00 - Valencia: 458218.00
VARIABLE: Age_0-4_Ptge Total outliers: 28 (0.34%) Límites IQR (1.5x): [-3.33, 9.25] Ejemplos MÁS ALTOS: - Lantz: 13.24 - Valle de Egúés / Eguesibar: 12.86 - Renau: 11.92	VARIABLE: UnemployLess25_Ptge Total outliers: 225 (2.77%) Límites IQR (1.5x): [-15.70, 26.17] Ejemplos MÁS ALTOS: - Huerta de la Obispalía: 100.00 - Escamilla: 100.00 - Zorita de los Canes: 100.00	VARIABLE: ServicesUnemploymentPtge Total outliers: 702 (8.65%) Límites IQR (1.5x): [16.80, 105.33] Ejemplos MÁS BAJOS: - Zarzosa de Río Pisuerga: 0.00 - Fuente la Reina: 0.00 - Herbés: 0.00	VARIABLE: Pob2010 Total outliers: 1142 (14.07%) Límites IQR (1.5x): [-3281.00, 5941.00] Ejemplos MÁS ALTOS: - Madrid: 3273049.00 - Barcelona: 1619337.00 - Valencia: 809267.00
VARIABLE: Age_over65_pct Total outliers: 59 (0.73%) Límites IQR (1.5x): [-5.70, 62.54] Ejemplos MÁS ALTOS: - Membibre de la Hoz: 76.47 - Cereza de Peñahorcada: 75.31 - Valsalobre: 75.00	VARIABLE: Unemploy25_40_Ptge Total outliers: 1370 (16.88%) Límites IQR (1.5x): [1.43, 73.81] Ejemplos MÁS ALTOS: - Villanueva de Carazo: 100.00 - Castellfort: 100.00 - Espadilla: 100.00	VARIABLE: totalEmpresas Total outliers: 1198 (14.76%) Límites IQR (1.5x): [-203.00, 357.00] Ejemplos MÁS ALTOS: - Madrid: 299397.00 - Barcelona: 174209.00 - Valencia: 62013.00	VARIABLE: SUPERFICIE Total outliers: 723 (8.91%) Límites IQR (1.5x): [-5743.69, 14477.40] Ejemplos MÁS ALTOS: - Cáceres: 175022.91 - Librilla: 167520.39 - Badajoz: 144037.48
VARIABLE: WomanPopulationPtge Total outliers: 497 (6.12%) Límites IQR (1.5x): [39.31, 56.41] Ejemplos MÁS ALTOS: - Arenas del Rey: 72.68 - Monasterio: 69.23 - Villarejo de la Peña: 66.67	VARIABLE: UnemployMore40_Ptge Total outliers: 1261 (15.54%) Límites IQR (1.5x): [13.55, 95.20] Ejemplos MÁS ALTOS: - Argelita: 100.00 - Castell de Cabres: 100.00 - Chodos / Xodos: 100.00	VARIABLE: Industria Total outliers: 1056 (13.01%) Límites IQR (1.5x): [-21.00, 35.00] Ejemplos MÁS ALTOS: - Madrid: 10521.00 - Barcelona: 5700.00 - Valencia: 2237.00	VARIABLE: PobChange_pct Total outliers: 372 (4.58%) Límites IQR (1.5x): [-26.13, 15.82] Ejemplos MÁS ALTOS: - Hornillos de Cameros: 138.46 - Yebeas: 136.91 - Moriscos: 91.35
VARIABLE: ForeignersPtge Total outliers: 361 (4.45%) Límites IQR (1.5x): [-9.16, 19.59] Ejemplos MÁS ALTOS: - Rojales: 71.47 - Llíbar: 70.70 - Torre del Burgo: 68.86	VARIABLE: AgricultureUnemploymentPtge Total outliers: 519 (6.30%) Límites IQR (1.5x): [-17.61, 29.35] Ejemplos MÁS ALTOS: - Torreblanca: 0.00 - Fresneda de la Sierra Tirón: 0.00 - Fuentebureba: 0.00	VARIABLE: ComercioHosteleria Total outliers: 1151 (14.18%) Límites IQR (1.5x): [-97.50, 162.50] Ejemplos MÁS ALTOS: - Madrid: 80856.00 - Barcelona: 51228.00 - Valencia: 20133.00	VARIABLE: PersonasInmueble Total outliers: 5 (0.06%) Límites IQR (1.5x): [-0.47, 3.05] Ejemplos MÁS ALTOS: - La Mojonera: 3.33 - Gurb: 3.21 - Rivas-Vaciamadrid: 3.21
VARIABLE: SameComAutonPtge Total outliers: 269 (3.31%) Límites IQR (1.5x): [53.83, 112.44] Ejemplos MÁS BAJOS: - Illán de Vacas: 0.00 - El Viso de San Juan: 15.41 - Ugena: 15.75	VARIABLE: AgriculturaUnemploymentPtge Total outliers: 519 (6.30%) Límites IQR (1.5x): [-17.61, 29.35] Ejemplos MÁS ALTOS: - Torreblanca: 0.00 - Fresneda de la Sierra Tirón: 0.00 - Fuentebureba: 0.00	VARIABLE: ComercioHosteleria Total outliers: 1151 (14.18%) Límites IQR (1.5x): [-97.50, 162.50] Ejemplos MÁS ALTOS: - Madrid: 80856.00 - Barcelona: 51228.00 - Valencia: 20133.00	VARIABLE: Exploraciones Total outliers: 798 (9.83%) Límites IQR (1.5x): [-137.00, 287.00] Ejemplos MÁS ALTOS: - El Ejido: 4759.00 - Murcia: 4006.00 - Lorca: 3511.00
VARIABLE: SameComAutonPtge Total outliers: 269 (3.31%) Límites IQR (1.5x): [53.83, 112.44] Ejemplos MÁS BAJOS: - Illán de Vacas: 0.00 - El Viso de San Juan: 15.41 - Ugena: 15.75	VARIABLE: AgriculturaUnemploymentPtge Total outliers: 519 (6.30%) Límites IQR (1.5x): [-17.61, 29.35] Ejemplos MÁS ALTOS: - Torreblanca: 0.00 - Fresneda de la Sierra Tirón: 0.00 - Fuentebureba: 0.00	VARIABLE: ComercioHosteleria Total outliers: 1151 (14.18%) Límites IQR (1.5x): [-97.50, 162.50] Ejemplos MÁS ALTOS: - Madrid: 80856.00 - Barcelona: 51228.00 - Valencia: 20133.00	VARIABLE: Age_5_10_Ptge Total outliers: 2 (0.02%) Límites IQR (1.5x): [-5.12, 26.49] Ejemplos MÁS ALTOS: - Castilleja de Guzmán: 28.13 - Cizur: 26.68

Tras analizar detalladamente, se toma la decisión de la no eliminación de valores atípicos, justificando en los siguientes puntos el porqué:

- Municipios muy pequeños (<2500 habitantes) vs grandes ciudades (>1M habitantes).
- Zonas rurales vs urbanas con características socioeconómicas muy diferentes.
- Un municipio con alto % de población extranjera no es un error, es un caso real (ej: zonas turísticas).
- Municipios donde el 100% de los votos van a la izquierda o derecha.
- Municipios con 100% de población en actividad laboral.

Estamos analizando datos electorales de municipios españoles. Esta heterogeneidad es real y relevante para el análisis, por mucho que estadísticamente puedan parecer valores disparatados. Eliminar estos casos supondría la pérdida de información sobre la diversidad territorial española. Además, los modelos de regresión lineal y logística son relativamente robustos a atípicos cuando el tamaño muestral es grande, las variables categóricas (“CCAA”, “Densidad”) capturarán parte de esta heterogeneidad estructural y, por último, la validación cruzada nos permitirá evaluar si los atípicos afectan la generalización.

6. Análisis de valores perdidos

El análisis muestra que varias variables presentan valores nulos. Debido a la variabilidad en el número de valores nulos, se decide implementar una estrategia diferenciada para los distintos casos:

```
# Identificamos columnas con valores nulos
null_columns = df.columns[df.isnull().any()]

null_summary = pd.DataFrame(index=null_columns, columns=['Null Count', 'Null Percentage'])

for col in null_columns:
    null_summary.loc[col, 'Null Count'] = df[col].isnull().sum()
    null_summary.loc[col, 'Null Percentage'] = (df[col].isnull().sum() / len(df)) * 100

null_summary = null_summary.sort_values(by='Null Count', ascending=False)
null_summary
```

	Null Count	Null Percentage
ForeignersPtge	653	8.044844
Explotaciones	189	2.328446
Industria	188	2.316127
Construccion	139	1.712455
inmuebles	138	1.700136
PersonasInmueble	138	1.700136
Densidad	92	1.133424
Servicios	62	0.763829
ComercTTEHosteleria	9	0.110878
SUPERFICIE	8	0.098559
Pob2010	7	0.086239
PobChange_pct	7	0.086239
totalEmpresas	5	0.061599
Age_over65_pct	3	0.036959
SameComAutonPtge	3	0.036959

Se decide la eliminación de registros para las variables con menos de 10 valores nulos para mantener la integridad de los datos, y la imputación por mediana (medida más robusta que la media, ya que esta no se ve afectada por valores atípicos, y estamos ante una distribución asimétrica en la que hemos decidido conservar esos valores atípicos) para las variables con más de 10 valores nulos. La variable “Densidad” es una excepción a la que se decide reemplazar los valores nulos por la moda, ya que es una variable categórica. Posteriormente, comprobamos que no quedan valores perdidos con la función ‘df.isnull().sum()’.

```
eliminar_nulos = list(null_summary.loc[null_summary['Null Count'] < 10].index)

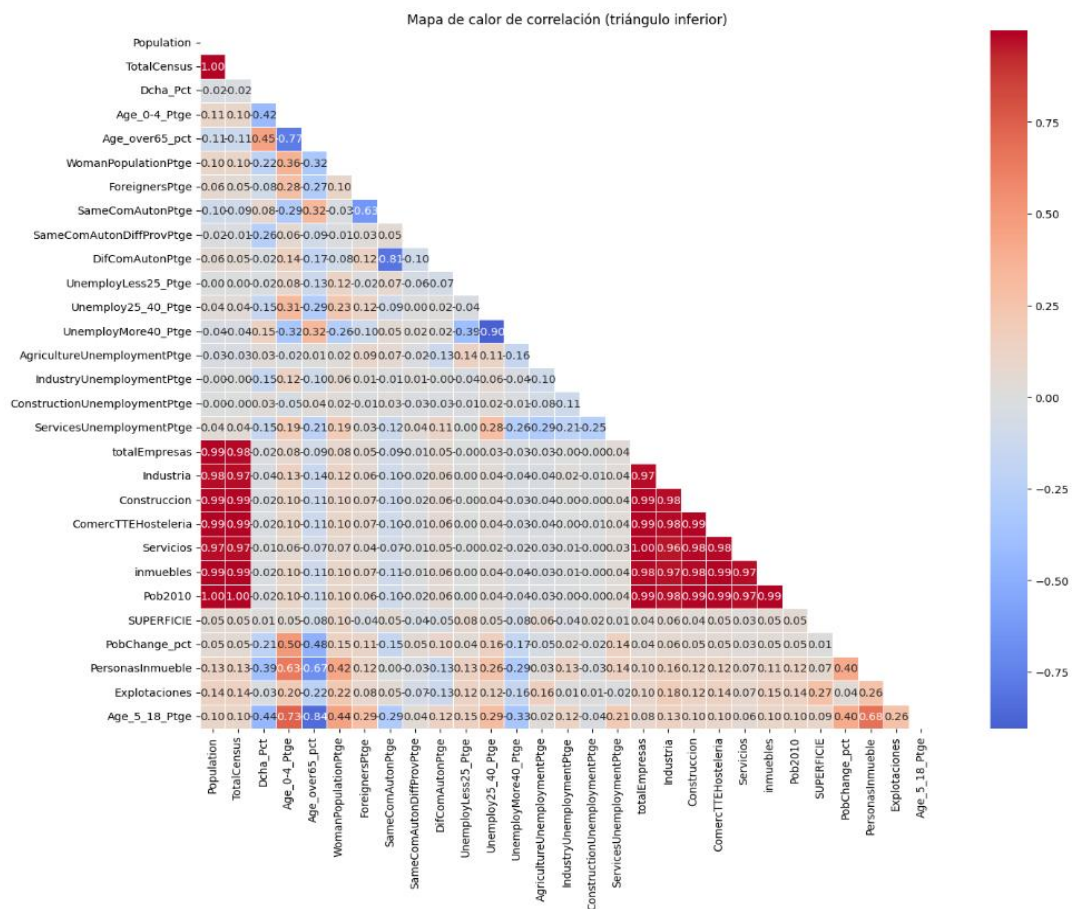
df = df.dropna(subset=eliminar_nulos)

nulos_a_imputar = list(null_summary.loc[null_summary['Null Count'] > 10].index)
nulos_a_imputar.remove('Densidad')

for var in nulos_a_imputar:
    mediana = df[var].median()
    df[var] = df[var].fillna(mediana)

df['Densidad'] = df['Densidad'].fillna(df['Densidad'].mode()[0])
```

7. Detección de las relaciones entre las variables



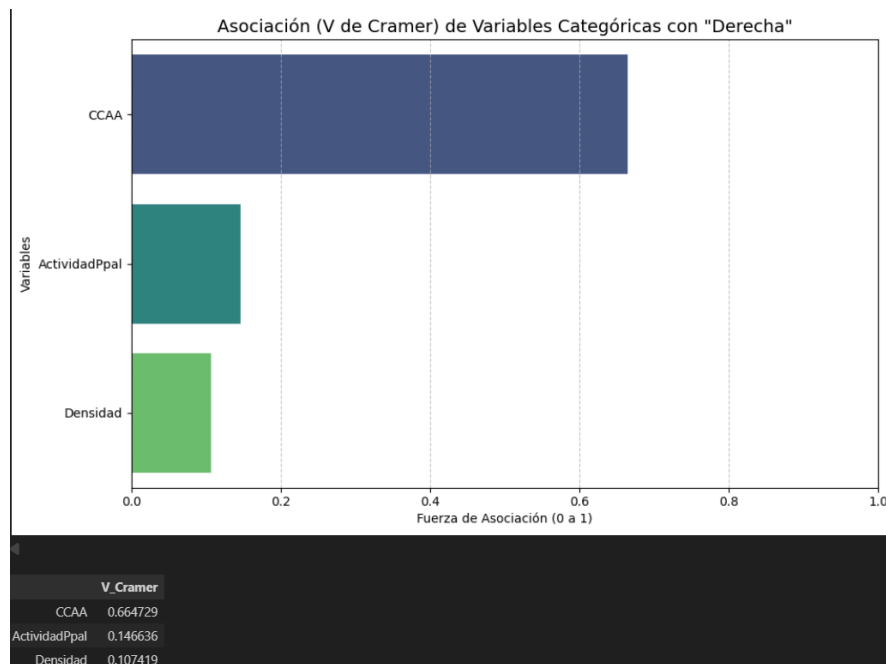
Tras analizar la matriz de correlación, identificamos varios grupos de variables con correlaciones muy altas (>0.90) marcadas en rojo. Estas altas correlaciones pueden derivar en el problema de la multicolinealidad, por lo que hay que eliminar las variables redundantes manteniendo aquellas que sean más recientes o generales, aporten información única o tengan mejor calidad de datos:

Variables eliminadas y justificación:

1. Pob2010 (correlación 1.00 con Population)
 - Razón: Ambas miden lo mismo (población) en años muy cercanos
 - Decisión: Mantener Population (más reciente) y eliminar Pob2010
2. TotalCensus (correlación 1.00 con Population)
 - Razón: El censo total es prácticamente idéntico a la población
 - Decisión: Mantener Population como variable más general
3. inmuebles (correlación 0.99 con Population)
 - Razón: El número de inmuebles es directamente proporcional a la población
 - Decisión: Mantener PersonasInmueble (ratio) que aporta más información que el valor absoluto
4. Industria, Construcción, Comercio, Hostelería, Servicios (correlación > 0.95 con totalEmpresas)
 - Razón: Son componentes del total de empresas, por lo que están perfectamente correlacionadas
 - Decisión: Mantener 'ActividadPpal' (variable categórica que indica el sector predominante) que captura la información relevante sin multicolinealidad

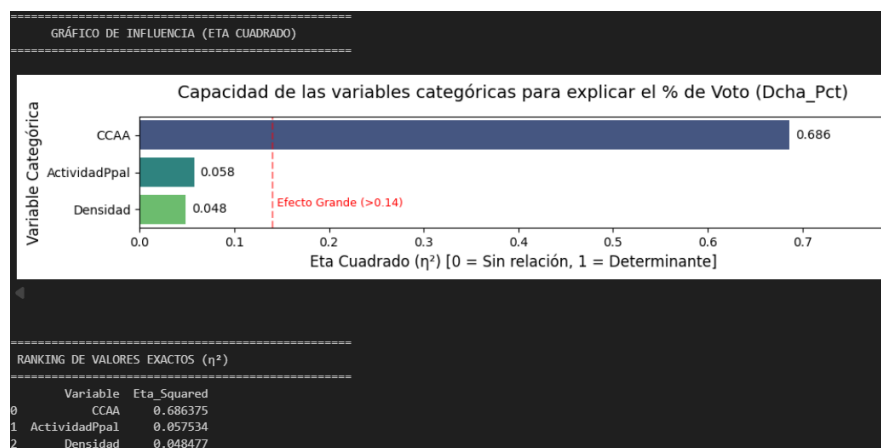
Además de la multicolinealidad, la matriz de confusión nos permite observar las relaciones lineales de la variable objetivo 'Dcha_Pct' con el resto de las variables numéricas. Por ejemplo, para las variables de los porcentajes de personas por edad, nos damos cuenta de que existen correlaciones moderadas, siendo negativas excepto para las personas mayores de 65. Esto nos indica que las personas más jóvenes están más inclinadas hacia la izquierda, y las personas mayores de 65 años hacia la derecha. También existe una ligera correlación negativa entre el porcentaje de votos a la derecha y el porcentaje de población femenina, mostrando que las mujeres se ven más inclinadas hacia políticas de izquierdas. Por último, también destacamos una ligera correlación de la variable objetivo con 'PersonasInmueble', quizá asociado a densidad o urbanización.

Por otro lado, un análisis mediante la V de Cramer mide la asociación entre variables categóricas para nuestra variable objetivo ‘Derecha’, en un rango entre 0 y 1, siendo 0 una asociación nula entre variables y 1 una asociación perfecta.



Para la variable objetivo ‘Derecha’ se observa una relación medianamente alta con ‘CCAA’ (valor de 0.665). El resto de las variables categóricas, ‘ActividadPpal’ y ‘Densidad’, poseen valores bajos de relación.

Por último, para medir la influencia de las variables categóricas sobre nuestra variable objetivo numérica ‘Dcha_Pct’, la métrica estadística correcta es la Eta Cuadrado.



Los resultandos son similares a los de la variable ‘Derecha’, hay una relación evidente entre ‘Dcha_Pct’ y ‘CCAA’ con un Eta Cuadrado de 0.686. Las variables ‘ActividadPpal’ y ‘Densidad’ las podríamos considerar ruido.

8. Construcción del modelo de regresión lineal

```
data = df.copy()
X = data.drop(columns=['Dcha_Pct', 'Derecha'], axis=1)
y = data['Dcha_Pct']
var_cont = X.select_dtypes(include=['float64', 'int64']).columns.tolist()
var_categ = X.select_dtypes(include=['object']).columns.tolist()
```

Este primer paso para la regresión lineal consiste en, inicialmente, eliminar la variable binaria ('Derecha'), ya que es la variable objetivo de la regresión logística. Además, se separa la variable objetivo 'Dcha_Pct' del resto de variables predictoras. También se diferencia las variables categóricas de las continuas.

Una vez finalizado el primer paso, se hace una división del dataset en train y test (80% y 20% respectivamente), y posteriormente, procedemos a la construcción de modelos de regresión lineal basados en los métodos de selección de variables clásicos con los criterios AIC y BIC, estos son:

- Método Backward: Comienza con todas las variables en el modelo y va eliminando gradualmente las menos relevantes. En cada iteración, se prueba eliminar una de ellas y se observa el rendimiento del modelo con y sin ella. Si al eliminarla el modelo no es significativamente peor, se descarta. Una vez descartada una variable, no puede volver a entrar al modelo.
- Método Forward: Comienza con el modelo vacío y agrega variables progresivamente. Se inicia con la variable más relevante, y en cada iteración se añade la que más mejora al rendimiento del modelo hasta alcanzar un conjunto óptimo de variables. Una vez añadida una variable al modelo, no puede salir.
- Método Stepwise: Este método combina los métodos anteriores, agrega variables que mejoran el rendimiento y elimina las menos significativas en cada iteración.
- AIC (Akaike Information Criterion): Se orienta a la capacidad predictiva, buscando el modelo que minimice la pérdida de información sin importar si el modelo es el "verdadero". Su penalización es constante, lo que lo hace más permisivo con la complejidad y propenso a incluir variables que mejoren el ajuste.
- BIC (Bayesian Information Criterion): Se basa en la probabilidad bayesiana y busca identificar el modelo real que generó los datos bajo un principio de parsimonia. Al penalizar en función del tamaño de la muestra, es más estricto que el AIC y tiende a seleccionar modelos más simples y robustos.

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1234567)

modeloStepAIC = lm_stepwise(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloStepBIC = lm_stepwise(y_train, x_train, var_cont, var_categ, [], 'BIC')
modeloBackAIC = lm_backward(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloBackBIC = lm_backward(y_train, x_train, var_cont, var_categ, [], 'BIC')
modeloForwAIC = lm_forward(y_train, x_train, var_cont, var_categ, [], 'AIC')
modeloForwBIC = lm_forward(y_train, x_train, var_cont, var_categ, [], 'BIC')

✓ lm 31.7s

Start: AIC = 57157.36347455925

y ~ 1

Variable      AIC
+ CCAA 49726.618590
+ Age_over65_pct 55650.002859
+ Age_5_18_Ptge 55731.047953
+ Age_0-4_Ptge 55872.552732
+ PersonasInmueble 56066.907241
+ SameComAutonDiffProvPtge 56697.972962
+ ActividadPpal 56763.012746
+ WomanPopulationPtge 56828.982426
+ Densidad 56834.246350
+ PobChange_pct 56838.193853
+ ServicesUnemploymentPtge 57001.380075
```

Los modelos ya entrenados, se evaluaron tanto en train como en test mediante la métrica del R^2 , dejando los siguientes resultados:

Método	Métrica	R^2 Train	R^2 Test	Nº Parámetros
Backward	AIC	0.710825	0.711900	24
Backward	BIC	0.709171	0.708783	16
Forward	AIC	0.710130	0.710747	23
Forward	BIC	0.708430	0.707767	15
Stepwise	AIC	0.710130	0.710747	23
Stepwise	BIC	0.708430	0.707767	15

En un principio los modelos con criterio ‘BIC’ ofrecen resultados más óptimos debido a que hacen uso de un menor número variables que los modelos ‘AIC’. Aunque las métricas de R^2 son muy similares, el hecho de utilizar un menor número de variables, nos ofrece mejores resultados en términos parsimonia-rendimiento. No obstante, estos resultados no son suficientes para determinar un modelo ganador, por lo que sometemos a los modelos a un proceso de validación cruzada de 40 iteraciones, ya que ofrece más seguridad para garantizar la robustez de los modelos, aunque suponga una mayor carga computacional.

```

lista_modelos = {}
if 'modelos_generados' in locals():
    lista_modelos = modelos_generados.copy()
else:
    nombres_probables = ['modeloStepAIC', 'modeloStepBIC', 'modeloBackAIC', 'modeloBackBIC', 'modeloForwAIC', 'modeloForwBIC']
    for nombre in nombres_probables:
        if nombre in locals():
            lista_modelos[nombre] = locals()[nombre]

if not lista_modelos:
    print("No se encontraron modelos para validar. Asegúrate de ejecutar primero la selección de modelos.")
else:
    modelos_unicos = {}
    for nombre, modelo_res in lista_modelos.items():
        v_cont = tuple(sorted(modelo_res['Variables']['cont']))
        v_cat = tuple(sorted(modelo_res['Variables']['categ']))
        v_inter = tuple(sorted(modelo_res['Variables']['inter'])) if 'inter' in modelo_res['Variables'] else ()
        firma = (v_cont, v_cat, v_inter)

        if firma not in modelos_unicos:
            modelos_unicos[firma] = {
                'Nombre': nombre,
                'ModeloRes': modelo_res
            }

    # Bucle de Validación Cruzada (40 repeticiones x 5 folds)
    if 'y_train' not in locals() or 'x_train' not in locals():
        print("Error: 'y_train' o 'x_train' no están definidos.")
    else:
        results = pd.DataFrame({'Rsquared': [], 'Adj_Rsquared': [], 'Resample': [], 'Modelo': [], 'N_Params': []})
        n_repeticiones = 40
        n_folds = 5
        n_muestras = len(y_train)
        n_test_fold = n_muestras / n_folds # Aproximado

        print(f"\nIniciando validación cruzada para {len(modelos_unicos)} modelos únicos ({n_repeticiones} repeticiones de {n_folds}-fold CV)...")

        for rep in range(n_repeticiones):
            if (rep + 1) % 5 == 0:
                print(f"Repetición {rep + 1}/{n_repeticiones}...")

            results_rep = pd.DataFrame()

            for firma, info in modelos_unicos.items():
                nombre_mod = info['Nombre']
                vars_mod = info['ModeloRes']['Variables']
                n_params = len(info['ModeloRes']['Modelo'].params)

                r2_scores = validation_cruzada_lm(
                    n_folds,
                    x_train,
                    y_train,
                    vars_mod['cont'],
                    vars_mod['categ'],
                    vars_mod.get('inter', [])
                )

                # Calcular R2 Ajustado manualmente para cada fold
                # Adj R2 = 1 - (1 - R2) * (n - 1) / (n - p - 1)
                adj_r2_scores = [1 - (1 - r2) * (n_test_fold - 1) / (n_test_fold - n_params - 1) for r2 in r2_scores]

                df_temp = pd.DataFrame({
                    'Rsquared': r2_scores,
                    'Adj_Rsquared': adj_r2_scores,
                    'Resample': [f'Rep{rep + 1}'] * n_folds,
                    'Modelo': [nombre_mod] * n_folds,
                    'N_Params': [n_params] * n_folds
                })

                results_rep = pd.concat([results_rep, df_temp], axis=0)

            results = pd.concat([results, results_rep], axis=0)

        print("\nValidación cruzada completada.")

```

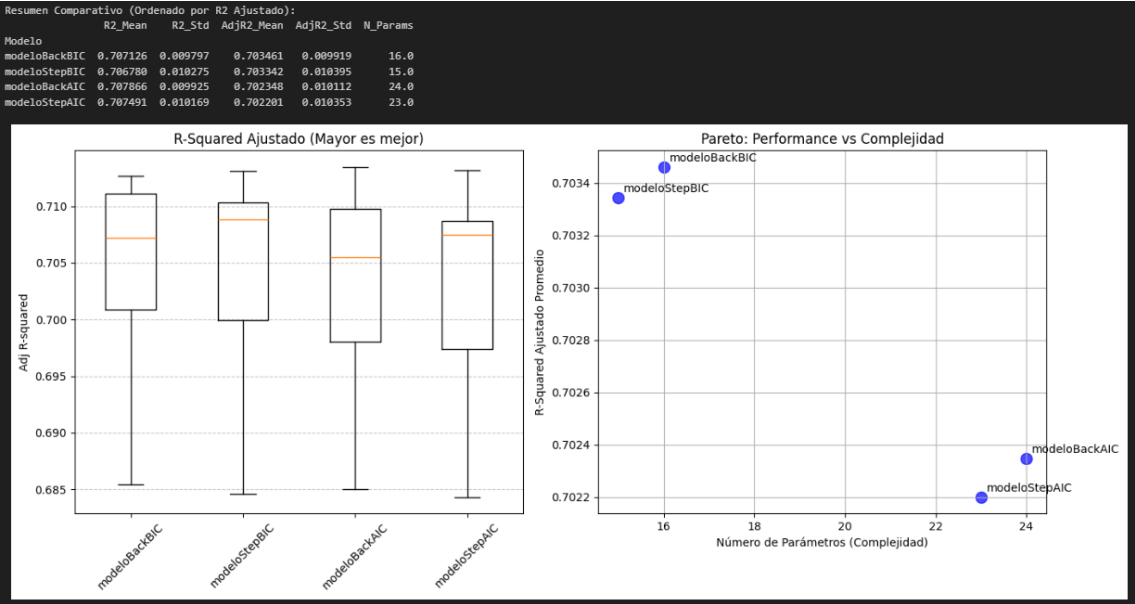
```

Iniciando validación cruzada para 4 modelos únicos (40 repeticiones de 5-fold CV)...
Repetición 5/40...
Repetición 10/40...
Repetición 15/40...
Repetición 20/40...
Repetición 25/40...
Repetición 30/40...
Repetición 35/40...
Repetición 40/40...

Validación cruzada completada.

```

Mostramos lo resultados de este proceso:



Consideramos que los resultados de modelo StepwiseBIC son los más robustos y óptimos en términos parsimonia-rendimiento, y lo elegimos como modelo ganador.

OLS Regression Results						
Dep. Variable:	dcha_pct	R-squared:	0.708			
Model:	OLS	Adj. R-squared:	0.708			
Method:	Least Squares	F-statistic:	1122.			
Date:	Thu, 05 Feb 2026	Prob (F-statistic):	0.00			
Time:	12:07:54	Log-Likelihood:	-24586.			
No. Observations:	6478	AIC:	4.920e+04			
Df Residuals:	6463	BIC:	4.930e+04			
Df Model:	14					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	40.2011	0.876	45.916	0.000	38.485	41.917
Age_over65_pct	0.2801	0.016	17.358	0.000	0.248	0.312
ForeignersPtge	0.1917	0.021	9.199	0.000	0.151	0.233
ServicesUnemploymentPtge	-0.0450	0.006	-7.628	0.000	-0.057	-0.033
IndustryUnemploymentPtge	-0.0437	0.011	-3.845	0.000	-0.066	-0.021
CCAA_CV-Bal-Ast-Ext-IC	4.7506	0.575	8.267	0.000	3.624	5.877
CCAA_CastillaLeón	19.9619	0.563	35.479	0.000	18.859	21.065
CCAA_Cat-PV	-27.7185	0.588	-47.107	0.000	-28.872	-26.565
CCAA_Gal-CM-Ara	10.8176	0.546	19.812	0.000	9.747	11.888
CCAA_Navarra	-6.0747	0.888	-6.840	0.000	-7.816	-4.334
CCAA_Rio-Mur-Can-Mad	15.8685	0.705	22.493	0.000	14.485	17.252
ActividadPpal_Otro	0.0680	0.386	0.176	0.860	-0.688	0.824
ActividadPpal_Servicios	3.5101	0.582	6.032	0.000	2.369	4.651
Densidad_Baja	-2.3814	0.648	-3.677	0.000	-3.651	-1.112
Densidad_MuyBaja	-3.7161	0.631	-5.885	0.000	-4.954	-2.478
Omnibus:	122.288	Durbin-Watson:	1.988			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	250.754			
Skew:	-0.051	Prob(JB):	3.54e-55			
Kurtosis:	3.958	Cond. No.	677.			

TABLA DE SIGNIFICANCIA (Ordenada)						
Códigos:	**** < 0.001, *** < 0.01, ** < 0.05, * < 0.1					
	Coefficiente	Std. Error	t-value	P-Valor		
const	40.201099	0.875539	45.915816	0.000000e+00		
CCAA_Cat-PV	-27.718461	0.588412	-47.107269	0.000000e+00		
CCAA_CastillaLeón	19.961941	0.562648	35.475541	4.560732e-252		
CCAA_Rio-Mur-Can-Mad	15.868504	0.705409	22.492590	6.234568e-188		
CCAA_Gal-CM-Ara	10.817618	0.546010	19.812110	7.416118e-85		
Age_over65_pct	0.280070	0.016135	17.358449	5.253082e-66		
ForeignersPtge	0.191744	0.020844	9.199023	4.784503e-20		
CCAA_CV-Bal-Ast-Ext-IC	4.750618	0.574653	8.266937	1.653146e-16		
ServicesUnemploymentPtge	-0.045038	0.005904	-7.627818	2.731600e-14		
CCAA_Navarra	-6.074706	0.888157	-6.839676	8.665826e-12		
ActividadPpal_Servicios	3.510102	0.581953	6.031592	1.713222e-09		
Densidad_MuyBaja	-3.716113	0.631459	-5.884963	4.180842e-09		
IndustryUnemploymentPtge	-0.043689	0.011363	-3.844884	1.217678e-04		
Densidad_Baja	-2.381434	0.647589	-3.677387	2.375480e-04		
ActividadPpal_Otro	0.068002	0.385807	0.176259	8.600961e-01		
Signif.						
const	***					
CCAA_Cat-PV	***					
CCAA_CastillaLeón	***					
CCAA_Rio-Mur-Can-Mad	***					
CCAA_Gal-CM-Ara	***					
Age_over65_pct	***					
ForeignersPtge	***					
CCAA_CV-Bal-Ast-Ext-IC	***					
ServicesUnemploymentPtge	***					
CCAA_Navarra	***					
ActividadPpal_Servicios	***					
Densidad_MuyBaja	***					
IndustryUnemploymentPtge	***					
Densidad_Baja	***					
ActividadPpal_Otro						

Si echamos un vitazo a las variables continuas, ‘Age_over65_pct’ presenta un coeficiente estimado de 0.28 con una significancia estadística (t = 17.36). Esto indica una relación lineal positiva; por cada punto porcentual adicional de población mayor de 65 años en un municipio, el porcentaje de voto a la derecha se incrementa en 0.28%. Este hallazgo ratifica la hipótesis del comportamiento electoral conservador asociado al envejecimiento

poblacional, actuando como un estabilizador del voto de derecha. Por otra parte, 'ForeignersPtge' con un coeficiente de 0.1917, sugiere que la presencia de población extranjera moviliza el voto hacia la derecha. Específicamente, un incremento de un punto porcentual en la población extranjera se asocia con un aumento de 0.19% en el voto a partidos de derecha. Esto se puede deber a malas experiencias con partidos de izquierda o buenas experiencias con partidos de derecha en su país de origen.

En las variables categóricas, 'CCAA_Cat-PV', que agrupa a Cataluña y País Vasco, muestra el coeficiente más extremo y negativo del modelo (-27.72). En comparación con la categoría de referencia (intercepto), un municipio situado en estas comunidades autónomas tiene una esperanza matemática de voto a la derecha del 27.72% inferior. Una teoría es que estas CCAA son conocidas por el sentimiento nacionalista, y el modelo capturaría cómo esa identidad regional altera las dinámicas de voto que vemos a nivel nacional. En el extremo opuesto, 'CCAA_CastillaLeón' con 19.96, representa la CCAA más sólida para la variable dependiente. El modelo estima que, por el mero hecho de pertenecer a esta región, el porcentaje de voto a la derecha es casi 20% superior al de la región base. La magnitud de este coeficiente (35.48) subraya que la ubicación geográfica es, en este modelo, un predictor más determinante que las variables socioeconómicas. Además, las agrupaciones 'CCAA_Rio-Mur-Can-Mad' y 'CCAA_CM-Gal-Ara' con coeficientes de 15.87 y 10.82 dictaminan los municipios de estas agrupaciones inclinan el voto a la derecha >10-16%.

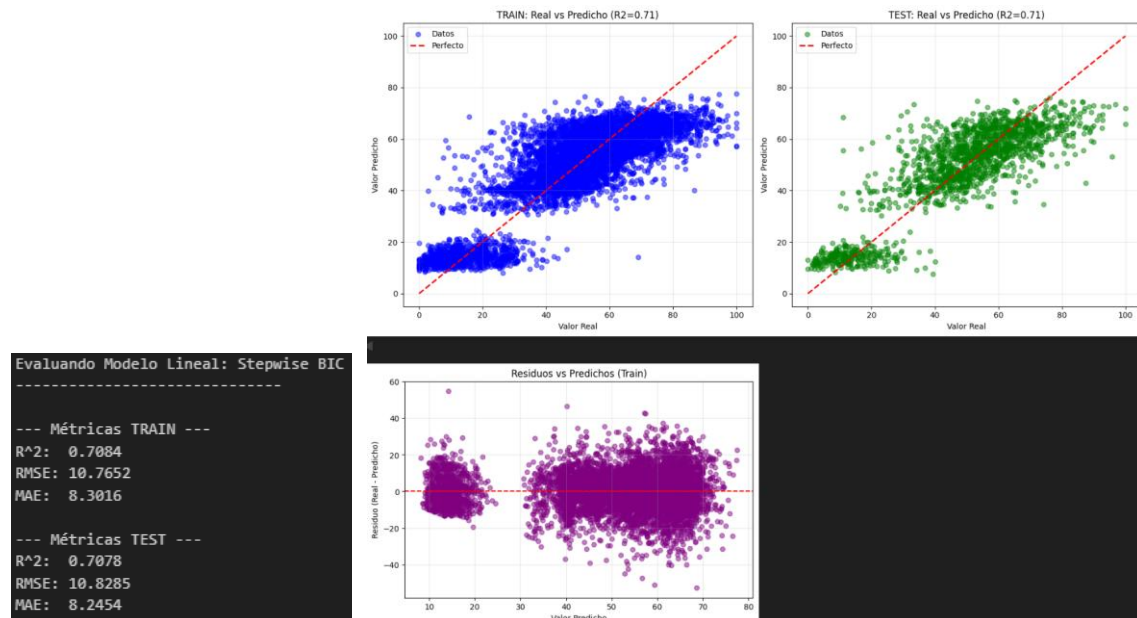
Este modelo se consolida como ganador debido a un equilibrio óptimo entre parsimonia y capacidad explicativa.

- Bondad de Ajuste (R^2): El modelo alcanza un R^2 de 0.708. En el contexto electoral, que contiene un alto componente impredecible, explicar el 70.8% de la varianza de la variable dependiente es un resultado bastante alto. El R^2 ajustado es idéntico (0.708), lo que confirma que no estamos inflando el ajuste artificialmente mediante la inclusión de variables irrelevantes.
- Significancia Global (Estadístico F): El valor $F = 1122$ con una $p\text{-value} = 0.00$ permite rechazar rotundamente la hipótesis nula global. Esto implica que el conjunto de covariables seleccionadas tiene una capacidad predictiva real y no es producto del azar.
- Robustez y Parsimonia: Con solo 14 grados de libertad utilizados para explicar 6,478 observaciones, el modelo se puede considerar bastante parsimonioso. La mayoría de los parámetros (excepto 'ActividadPpal_Otro') son significativos al 1% ($p < 0.01$), lo que demuestra que las variables seleccionadas son robustas.
- Diagnóstico de Multicolinealidad: El *Condition Number* es 677. Aunque técnicamente indica cierta multicolinealidad (esperable por la estructura de las agrupaciones regionales y variables demográficas), está lejos de los umbrales críticos (>1000) que harían inestables las estimaciones numéricas.

Aún así, el modelo puede ser mejorable, y estas son algunas recomendaciones para mejorar el modelo de manera manual:

- La variable 'ActividadPpal_Otro' no aporta significativamente ($p=0.86$). Eliminarla, simplifica la ecuación y evita que el modelo intente buscar patrones donde no los hay, dejando esa variabilidad en la constante base donde pertenece.
- Sustituir o complementar la densidad con la variable 'PobChange_pct' (Cambio de población) puede ser una buena idea. Un municipio puede tener baja densidad pero estar estable, mientras que otro puede estar perdiendo habitantes a gran velocidad. Los municipios en declive demográfico suelen desarrollar un voto de descontento o conservador más acentuado. Capturar la dinámica (si el pueblo se muere o crece) suele predecir mejor el voto que solo saber si es pequeño.
- El agrupamineto que hemos realizado en las categorías de la variable 'CCAA' puede no ser el más óptimo, necesitaríamos de un estudio más amplio y de distintas pruebas, para determinar cuál sería el agrupamineto idóneo de estas categorías. Por ejemplo, Navarra probablemente se podía haber agrupado en algún grupo y tener un mayor nivel de significación. También, cabe la probabilidad de que sea más óptimo para el modelo, que no hubiesen agrupaciones de CCAA, pese a que algunas de estas quedasen menos representadas. Al final, es lógico que Castilla y León tenga más representación que Murcia, ya que su población es bastante más extensa.

Vamos a evaluar el modelo ganador, y a graficar la regresión lineal con sus resultados.



En cuanto a los gráficos de Train y Test, lo primero que se percibe es que el modelo es robusto al pasar de train a test. Los resultados son prácticamente idénticos: el R^2 se queda en 0.708 y el error (RMSE) apenas cambia, lo que significa que el modelo no ha aprendido los datos de memoria (no hay *overfitting*) y sabe generalizar bien. Sin embargo, en ambos gráficos se aprecia que los puntos no forman una sola línea, sino que están separados en dos grupos. El modelo intenta pasar por el medio de todo, pero le cuesta un

poco entender por qué los datos están tan divididos, especialmente en los valores más altos donde los puntos se dispersan más.

Analizando los residuos observamos un valor del estadístico Durbin-Watson (1.988) muy cercano a 2, lo que sugiere residuos incorrelados. Atendiendo al gráfico se ve un hueco notable en el medio, entre el 20 y el 30 (de manera aproximada), lo que confirma que hay algo en los datos que el modelo lineal no está pillando, como si faltara una variable importante que de más sentido. Además, la "mancha" de puntos de la derecha es mucho más grande y ancha que la de la izquierda; esto nos dice que el modelo es bastante preciso cuando predice valores bajos, pero cuando intenta predecir valores altos se vuelve mucho más impreciso y falla por más margen. Aunque el modelo es estable, los residuos nos avisan de que una simple línea recta no es suficiente para explicar toda la historia de los datos, lo cual es comprensible teniendo en cuenta de que estamos hablando de votos en elecciones. El hecho de que varias personas vivan en el mismo sitio, tengan el mismo trabajo o compartan sus raíces no garantiza que voten igual. Esto demuestra que la intención de voto es algo mucho más complejo que solo el perfil de la persona.

9. Construcción del modelo de regresión logística

Al igual que en el caso de la regresión lineal, para la regresión logística, lo primero es eliminar en este caso la variable 'Dcha_Pct', ya que era la variable objetivo de la regresión lineal. Así mismo, como en el caso de la regresión lineal, se separa la variable objetivo 'Derecha' del resto de variables, se diferencian variables numéricas de categóricas, y se realiza la división entre los conjuntos de train y test. El código y métodos empleados fueron los mismos que para la regresión lineal, con la particularidad de que, en lugar de la función `lm_método`, se utilizó `glm_método` al tratarse de una regresión logística.

```
data_log = df.copy()
X_log = data_log.drop(columns=['Dcha_Pct', 'Derecha'], axis=1)
y_log = data_log['Derecha']

var_cont_log = X_log.select_dtypes(include=['float64', 'int64']).columns.tolist()
var_categ_log = X_log.select_dtypes(include=['object']).columns.tolist()
```

```
x_train_log, x_test_log, y_train_log, y_test_log = train_test_split(X_log, y_log, test_size=0.2, random_state=1234567)

modelologStepAIC = glm_stepwise(y_train_log, x_train_log, var_cont_log, var_categ_log, [], 'AIC')
modelologStepBIC = glm_stepwise(y_train_log, x_train_log, var_cont_log, var_categ_log, [], 'BIC')
modelologBackAIC = glm_backward(y_train_log, x_train_log, var_cont_log, var_categ_log, [], 'AIC')
modelologBackBIC = glm_backward(y_train_log, x_train_log, var_cont_log, var_categ_log, [], 'BIC')
modelologForwAIC = glm_forward(y_train_log, x_train_log, var_cont_log, var_categ_log, [], 'AIC')
modelologForwBIC = glm_forward(y_train_log, x_train_log, var_cont_log, var_categ_log, [], 'BIC')
```

✓ 13m 0.7s

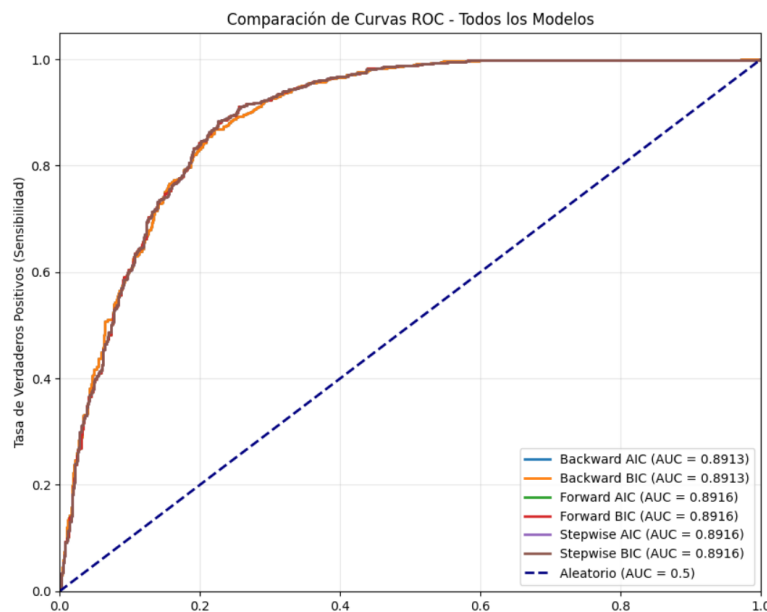
Start: AIC = 8580.051483964933

y ~ 1

Variable	AIC
+ CCAA	5368.354716
+ Age_over65_pct	7951.898018
+ Age_5_18_Ptge	7958.481709
+ PersonasInmueble	8080.629817
+ Age_0-4_Ptge	8103.497060
+ SameComAutonDiffProvPtge	8361.122313
+ ActividadPpal	8428.176962
+ WomanPopulationPtge	8460.375039
+ PobChange_pct	8466.894885
+ Densidad	8499.619761

Los modelos ya entrenados, se evaluaron tanto en train como en test, pero esta vez es AUC la métrica que se utiliza para comparar el rendimiento de los modelos:

Método	Métrica	AUC Train	AUC Test	Nº Parámetros
Backward	AIC	0.876356	0.891303	17
Backward	BIC	0.876356	0.891303	17
Forward	AIC	0.877932	0.891570	21
Forward	BIC	0.877932	0.891570	21
Stepwise	AIC	0.877918	0.891570	20
Stepwise	BIC	0.877918	0.891570	20



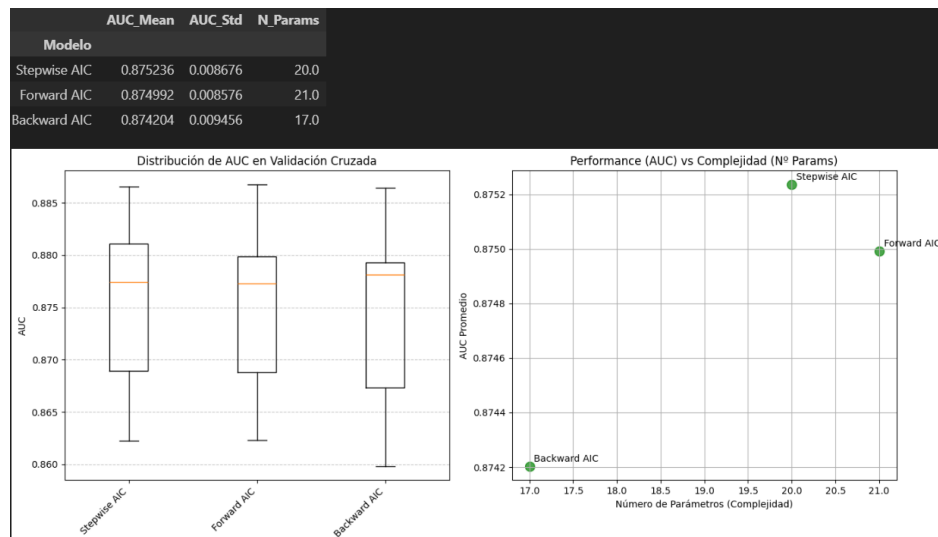
En el caso de la regresión logística, no hay diferencia entre los criterios AIC y BIC, ambos han obtenidos los mismos modelos en Backward, Forward y Stepwise. En un principio los modelos elegidos serán los de Backward, ya que aunque sus métricas son ligermante inferiores a las de Stepwise y Forward, son mejores en términos parsimonia-rendimiento al requerir un menor número de parámetros. No obstante, al igual que en la regresión lineal, esto no es suficiente para determinar un modelo ganador, por lo que sometemos también estos modelos a un proceso de regresión lineal de 40 iteraciones. El código es prácticamente idéntico al de regresión lineal.

```

Nota: 'Backward BIC' es idéntico a 'Backward AIC'. Se evaluará solo una vez.
Nota: 'Forward BIC' es idéntico a 'Forward AIC'. Se evaluará solo una vez.
Nota: 'Stepwise BIC' es idéntico a 'Stepwise AIC'. Se evaluará solo una vez.

Iniciando validación cruzada para 3 modelos únicos (40 reps de 5-fold CV)...
Repetición 5/40...
Repetición 10/40...
Repetición 15/40...
Repetición 20/40...
Repetición 25/40...
Repetición 30/40...
Repetición 35/40...
Repetición 40/40...

```



Los resultados de la validación cruzada respaldan lo que hemos comentado anteriormente, las métricas de Backward AIC son algo inferiores pero al requerir 3-4 variables menos, lo determinamos como modelo ganador.

El siguiente apartado, es realizar un análisis para determinar el punto de corte óptimo. Se va a utilizar el punto de corte de Youden, que, en regresión logística, es el umbral que maximiza la suma de sensibilidad y especificidad, buscando el mejor equilibrio entre aciertos en positivos y negativos, y se comparará con el punto de corte que maximiza el *accuracy*. Se usa para elegir el punto óptimo de clasificación en lugar del típico 0.5. Se realizó con el siguiente código:

```

modelo_ganador = modelologBackAIC
nombre_modelo = "Backward AIC"

# Preparar datos de test una sola vez
interacciones = modelo_ganador['Variables']['inter'] if 'inter' in modelo_ganador['Variables'] else []
x_test_modelo = crear_data_modelo(
    x_test_log,
    modelo_ganador['Variables']['cont'],
    modelo_ganador['Variables']['categ'],
    interacciones
)

# Obtener probabilidades (Predicciones continuas)
probs_test = modelo_ganador['Modelo'].predict_proba(x_test_modelo)[1:, 1]

cortes = np.arange(0.01, 1.0, 0.01)
resultados_corte = []

for c in cortes:
    preds = (probs_test > c).astype(int)

    tn, fp, fn, tp = confusion_matrix(y_test_log, preds).ravel()

    acc = (tp + tn) / (tp + tn + fp + fn)
    sens = tp / (tp + fn) if (tp + fn) > 0 else 0
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0
    youden = sens + spec - 1

    resultados_corte.append({
        'Corte': c,
        'Accuracy': acc,
        'Sensibilidad': sens,
        'Especificidad': spec,
        'Youden': youden
    })

df_metricas = pd.DataFrame(resultados_corte)

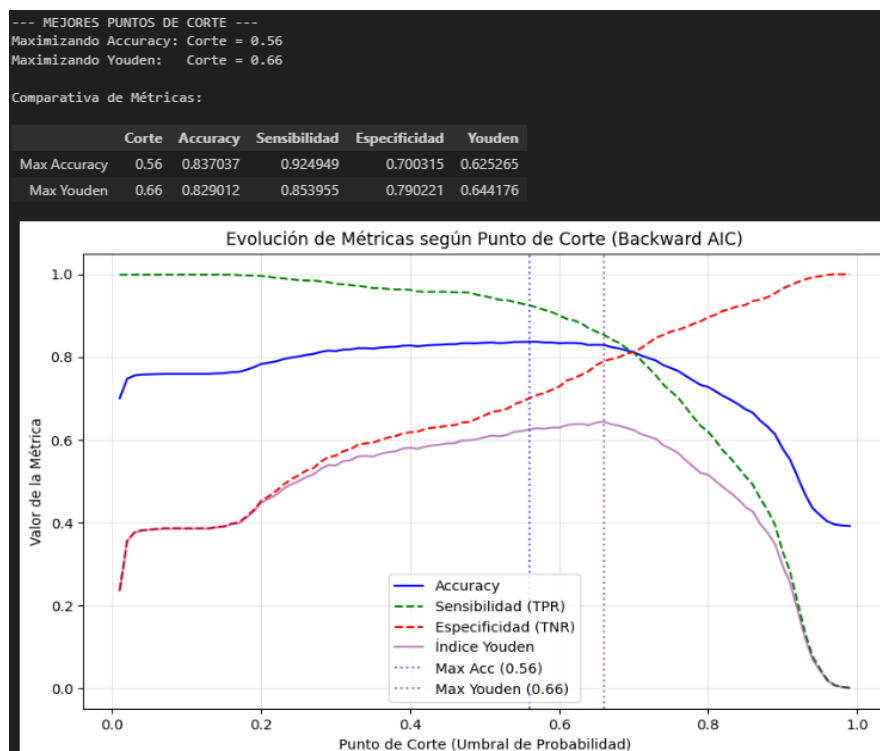
# Determinar los mejores puntos de corte
idx_max_acc = df_metricas['Accuracy'].idxmax()
idx_max_youden = df_metricas['Youden'].idxmax()

mejor_corte_acc = df_metricas.loc[idx_max_acc]
mejor_corte_youden = df_metricas.loc[idx_max_youden]

print("\n--- MEJORES PUNTOS DE CORTE ---")
print(f"Maximizando Accuracy: Corte = {mejor_corte_acc['Corte']:.2f}")
print(f"Maximizando Youden: Corte = {mejor_corte_youden['Corte']:.2f}")

# Mostrar métricas para esos puntos
resumen_cortes = pd.DataFrame([mejor_corte_acc, mejor_corte_youden])
resumen_cortes.index = ['Max Accuracy', 'Max Youden']
print("\nComparativa de Métricas:")
display(resumen_cortes)

```



Tras analizar el desempeño del clasificador, se opta por establecer el punto de corte en 0.66 (basado en el índice de Youden) frente al criterio de *Max Accuracy*. Aunque el umbral de 0.56 ofrece una *Accuracy* ligeramente superior (0.837), esta se logra a costa de un desequilibrio notable entre la Sensibilidad (0.925) y la Especificidad (0.7), lo que podría derivar en una sobreestimación sistemática del voto hacia la derecha.

Por el contrario, el punto de corte 0.66 maximiza el estadístico de Youden (0.644), garantizando un equilibrio más eficiente entre la tasa de verdaderos positivos y la capacidad de discriminación de los negativos. En un contexto electoral, esta decisión es preferible ya que reduce los falsos positivos y proporciona un modelo con mayor poder discriminante y robustez. En definitiva, se sacrifica apenas un 0.8% de precisión global a cambio de un incremento de 9% en la especificidad del modelo.

A continuación, vemos el cuadro resumen del modelo Backward AIC (modelo ganador), y comentamos sus resultados:

RESUMEN ESTADÍSTICO (Statsmodels Refit)				
Logit Regression Results				
Dep. Variable:	Derecha	No. Observations:	6478	
Model:	Logit	Df Residuals:	6461	
Method:	MLE	Df Model:	16	
Date:	Thu, 05 Feb 2026	Pseudo R-squ.:	0.4931	
Time:	12:26:39	Log-likelihood:	-2559.5	
converged:	True	LL-Null:	-4288.0	
Covariance Type:	nonrobust	LLR p-value:	0.000	
	coef	std err	z	P> z
const	3.1727	0.551	5.757	0.000
Age_over65_pct	0.0225	0.004	5.054	0.000
SameComAutonPtge	-0.0477	0.006	-8.286	0.000
SameComAutonDiffProvPtge	0.0296	0.009	3.350	0.001
DifComAutonPtge	-0.0644	0.007	-8.876	0.000
Explotaciones	0.0007	0.000	4.004	0.000
UnemployLess25_Ptge	-0.0121	0.003	-3.446	0.001
CCAA_CV-Bal-Ast-Ext-IC	1.7725	0.128	13.859	0.000
CCAA_CastillaLeón	3.4575	0.145	23.795	0.000
CCAA_Cat-PV	-4.4082	0.406	-10.870	0.000
CCAA_Gal-CM-Ara	2.2086	0.132	16.755	0.000
CCAA_Navarra	0.5226	0.194	2.700	0.007
CCAA_Rio-Mur-Can-Mad	3.3307	0.205	16.252	0.000
ActividadPpal_Otro	0.1318	0.101	1.299	0.194
ActividadPpal_Servicios	0.8608	0.192	4.492	0.000
Densidad_Baja	-0.1505	0.191	-0.787	0.431
Densidad_MuyBaja	-0.6364	0.182	-3.499	0.000

TABLA DE SIGNIFICANCIA (Ordenada por P-Valor)				
	Coefficiente	Std. Error	z-value	P-Valor
CCAA_CastillaLeón	3.457464	0.145301	23.795140	3.750058e-125
CCAA_Gal-CM-Ara	2.208585	0.131818	16.754782	5.225237e-63
CCAA_Rio-Mur-Can-Mad	3.330696	0.204941	16.251981	2.162628e-59
CCAA_CV-Bal-Ast-Ext-IC	1.772521	0.127900	13.858633	1.128018e-31
CCAA_Cat-PV	-4.408163	0.405545	-10.869724	1.606863e-27
DifComAutonPtge	-0.064405	0.007256	-8.875811	6.942576e-19
SameComAutonPtge	-0.047658	0.005808	-8.205955	2.287660e-16
const	3.172695	0.551090	5.757126	8.555794e-09
Age_over65_pct	0.022537	0.004459	5.054080	4.324702e-07
ActividadPpal_Servicios	0.860843	0.191649	4.491781	7.062988e-06
Explotaciones	0.000741	0.000185	4.004309	6.219915e-05
Densidad_MuyBaja	-0.636403	0.181857	-3.499461	4.662003e-04
UnemployLess25_Ptge	-0.012053	0.003498	-3.445856	5.692544e-04
SameComAutonDiffProvPtge	0.029567	0.008827	3.349520	8.095181e-04
CCAA_Navarra	0.522647	0.193588	2.699798	6.938167e-03
ActividadPpal_Otro	0.131765	0.101412	1.299306	1.938390e-01
Densidad_Baja	-0.150545	0.191368	-0.786677	4.314709e-01
Signif.				
CCAA_CastillaLeón	***			
CCAA_Gal-CM-Ara	***			
CCAA_Rio-Mur-Can-Mad	***			
CCAA_CV-Bal-Ast-Ext-IC	***			
CCAA_Cat-PV	***			
DifComAutonPtge	***			
SameComAutonPtge	***			
const	***			
Age_over65_pct	***			
ActividadPpal_Servicios	***			
Explotaciones	***			
Densidad_MuyBaja	***			
UnemployLess25_Ptge	***			
SameComAutonDiffProvPtge	***			
CCAA_Navarra	**			
ActividadPpal_Otro				
Densidad_Baja				
ODDS RATIOS (Exp(Coef))				
CCAA_CastillaLeón	31.736377			
CCAA_Rio-Mur-Can-Mad	27.957803			
const	23.871737			
CCAA_Gal-CM-Ara	9.102826			
CCAA_CV-Bal-Ast-Ext-IC	5.885674			
ActividadPpal_Servicios	2.365154			
CCAA_Navarra	1.686486			
ActividadPpal_Otro	1.140840			
SameComAutonDiffProvPtge	1.030008			
Age_over65_pct	1.022793			
Explotaciones	1.000741			
UnemployLess25_Ptge	0.988019			
SameComAutonPtge	0.953460			
DifComAutonPtge	0.937625			
Densidad_Baja	0.860239			
Densidad_MuyBaja	0.529193			
CCAA_Cat-PV	0.012178			
dtype: float64				

En las variables continuas, ‘DifComAutonPtge’ (porcentaje de población nacida en una comunidad autónoma distinta) y ‘Age_over65_pct’ (porcentaje de mayores de 65 años) son los predictores más robustos. La variable migratoria presenta un coeficiente negativo de -0.0644, lo que indica que en municipios con una mayor diversidad de origen poblacional nacional, la probabilidad de un resultado hacia la "Derecha" disminuye significativamente; específicamente, cada punto porcentual de incremento en esta diversidad reduce las posibilidades (odds) en un 6.2%. Por otro lado, el envejecimiento poblacional actúa como el contrapunto conservador del modelo. Con un coeficiente positivo de 0.0225, la variable ‘Age_over65_pct’ confirma que el perfil demográfico envejecido es un predictor positivo para la variable dependiente. Aunque el incremento por cada unidad porcentual parece pequeño (un 2.3% de aumento en los odds), su altísima significancia estadística ($z = 5.054$) sugiere que es un factor estructural muy estable en el comportamiento del modelo.

En cuanto a las variables categóricas, la procedencia geográfica domina el modelo, destacando ‘CCAA_CastillaLeón’ y ‘CCAA_Cat-PV’ por representar los dos extremos del espectro. Castilla y León se posiciona como el predictor positivo más potente de todo el estudio: el hecho de que una observación pertenezca a esta región multiplica por más de 31 veces las probabilidades de "Derecha" respecto a la categoría base. En el extremo opuesto, la agrupación de Cataluña y País Vasco (‘CCAA_Cat-PV’) muestra un coeficiente negativo de -4.4082, el más bajo del modelo. Esto se traduce en un Odds Ratio

de apenas 0.012, lo que significa que la probabilidad de un resultado de "Derecha" en estas comunidades es prácticamente nula (una reducción del 98.8%) en comparación con el grupo de referencia. Estos resultados subrayan que, en este modelo, la ubicación geográfica tiene mucho más peso que factores socioeconómicos como la densidad o el tipo de actividad principal.

El modelo de regresión logística ganador demuestra un equilibrio óptimo entre capacidad discriminante, parsimonia y coherencia teórica.

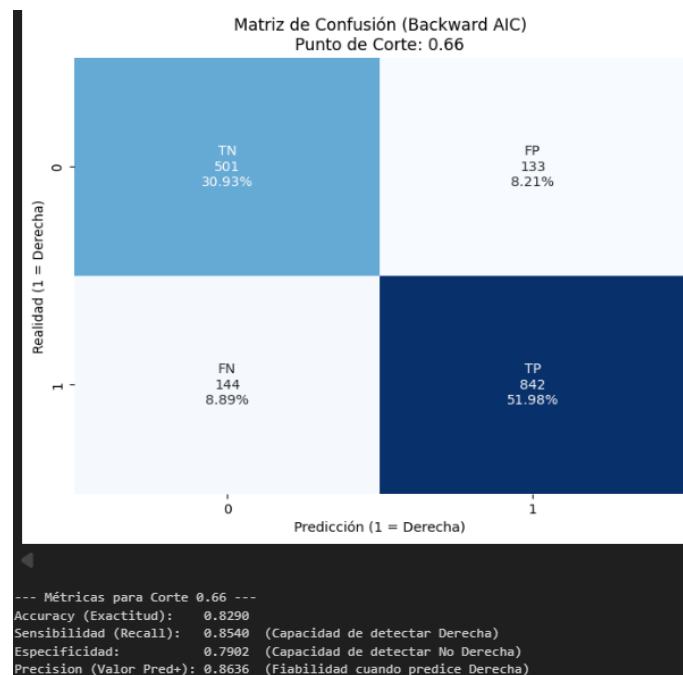
- **Bondad de Ajuste y Capacidad Explicativa:** El indicador más revelador de la calidad es el Pseudo R^2 de 0.4031. En el contexto de modelos de elección binaria, donde no existe un R^2 matemático equivalente al de OLS, un valor de Pseudo- R^2 entre 0.2 y 0.4 se considera indicativo de un ajuste excelente. Alcanzar un 0.403 en un estudio de comportamiento electoral implica que el modelo ha capturado con gran precisión la estructura latente que diferencia a los municipios de "Derecha" frente a los de "Izquierda". La reducción drástica de la Log-Likelihood desde el modelo nulo ($LL_{null} = -4288.0$) hasta el modelo ajustado (Log-Likelihood = -2559.5) confirma que la inclusión de los predictores reduce la incertidumbre del sistema de manera sustancial.
- **Significancia Global y Robustez Estadística:** LLR p-value arroja un valor de 0.000, lo que permite rechazar con total seguridad la hipótesis nula global. Esto certifica que el modelo conjunto es estadísticamente superior a un modelo vacío. El modelo utiliza 16 grados de libertad (covariables) para explicar un dataset de 6,478 observaciones. Esta relación (más de 400 observaciones por parámetro estimado) garantiza la estabilidad de los estimadores y minimiza el riesgo de *overfitting*. La convergencia del algoritmo de Máxima Verosimilitud (MLE) fue exitosa, lo que asegura que el modelo es numéricamente estable y que no hay problemas de colinealidad extrema que invaliden los resultados.
- El análisis de z-scores revela una especificación depurada y significativa. De las 17 variables (incluyendo la constante), 14 son significativas al nivel de confianza del 99% ($p < 0.001$). Esto incluye todas las variables geográficas principales, edad y estructura económica (explotaciones, desempleo). Solo 'ActividadPpal_Otro' y 'Densidad_Baja' no alcanzan significancia estadística.
- La fortaleza del modelo reside en su capacidad para cuantificar la heterogeneidad estructural del voto. Los Odds Ratios (OR) permiten establecer un ranking claro de importancia. Las variables con mayor tamaño son las regionales. El modelo captura exitosamente que el territorial es el parámetro principal de predicción.

Para elevar la calidad del modelo de regresión logística, se presentan una serie de propuestas que se podrían llevar a cabo para la optimización manual del modelo:

- Eliminar variables con una contribución marginal nula al poder explicativo, como 'Densidad_Baja' ($p > 0.4$) y 'ActividadPpal_Otro'. Mantener predictores no significativos solo incrementa la varianza de los estimadores y el riesgo de *overfitting*.

- La matriz de correlación revela una colinealidad de -0.80 entre 'SameComAutonPtge' y 'DifComAutonPtge'. Sería conveniente realizar un Factor de Inflación de la Varianza (VIF) con el objetivo de ver si los coeficientes están inflados, y por tanto, si eliminar una de las dos variables para evitar multicolinealidad.
- El agrupamineto que hemos realizado de las categorías en la variable 'CCAA' puede no ser el más óptimo. Misma explicación que en el apartado de regresión lineal.

Tras comentar los resultados, por último, vamos a comprobar la capacidad de nuestro modelo ganador (aplicando el punto de corte seleccionado) para predecir nuestra variable objetivo 'Derecha'. Visualizamos los resultados a través de una matriz de confusión:



Al aplicar el Índice de Youden para determinar el punto de corte óptimo (0.66), logramos un equilibrio robusto entre la sensibilidad (85.4%) y la especificidad (79%). Es notable que el umbral se sitúe por encima del estándar 0.5, lo que sugiere que el modelo requiere una mayor certidumbre probabilística para clasificar una observación como "Derecha", logrando así una precisión del 86.36% que minimiza el error de falsos positivos. Con una exactitud global del 82.9%, el modelo demuestra una alta capacidad discriminatoria; aunque existe un ligero sesgo hacia la detección de la clase positiva, el bajo porcentaje de falsos negativos (8.89%) confirma que la selección de variables mediante Backward AIC ha resultado en un clasificador con un excelente poder predictivo y generalización.